DBMS PROJECT • PRESENTATION 2 OF 3

Hostel Accommodation & Student Services Management System

Database Design, Sample Data & Query Execution

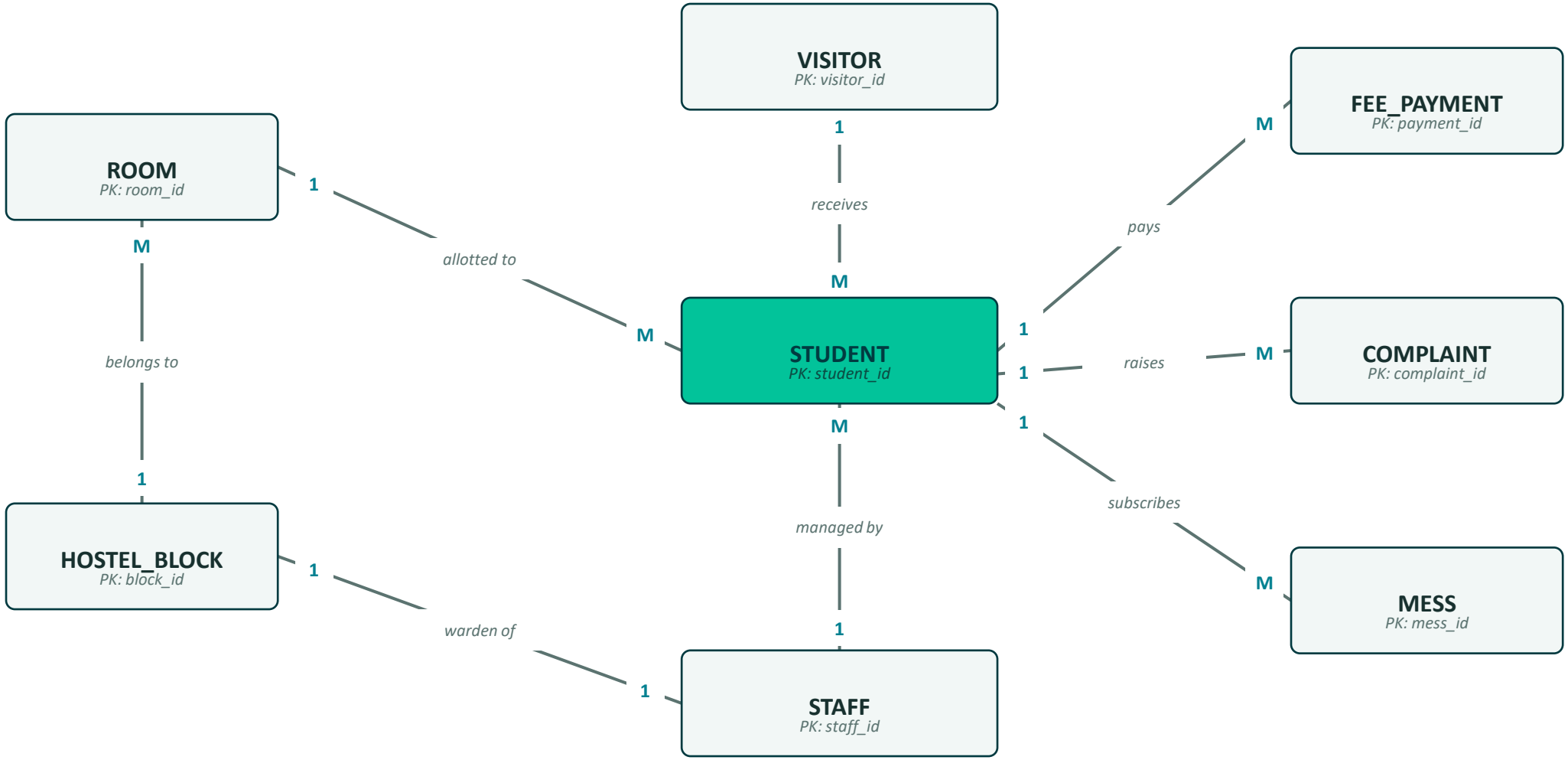
ER Diagram | DB Design | Filled Tables | Query Execution

Team Members: [Hussain-25WU0102100]

RDBMS: MySQL 8.0

Entity-Relationship Diagram

Primary keys, foreign keys, and cardinality for every relationship



Relational Schema

Normalized to 3NF — each table stores one entity, no repeating groups

Staff

PK staff_id

FK —

Hostel_Block

PK block_id

FK warden_id → Staff

Room

PK room_id

FK block_id → Hostel_Block

Student

PK student_id

FK room_id → Room

Fee_Payment

PK payment_id

FK student_id → Student

Complaint

PK complaint_id

FK student_id → Student

Mess

PK mess_id

FK student_id → Student

Visitor

PK visitor_id

FK student_id → Student

Normalization note: Room, Block, and Staff were split into separate tables (rather than repeated on every Student row) to remove transitive and partial dependencies — satisfying 2NF and 3NF.

Filled-in Tables

Sample data — Student and Room

Student

student_id	student_name	gender	room_id
1	Aditya Sharma	M	101
2	Priya Nair	F	101
3	Rahul Gupta	M	102
4	Sneha Iyer	F	201
5	Karan Mehta	M	201
6	Divya Singh	F	202
7	Arjun Rao	M	202
8	Meera Joshi	F	102

Room

room_id	room_no	block_id	status
101	A-101	1	Occupied
102	A-102	1	Occupied
103	A-103	1	Available
201	B-201	2	Occupied
202	B-202	2	Occupied

Filled-in Tables

Sample data — Fee_Payment and Complaint

Fee_Payment

payment_id	student_id	amount	status
1	1	45000.00	Paid
2	2	45000.00	Pending
3	3	45000.00	Paid
4	4	42000.00	Pending
5	5	42000.00	Paid
6	6	42000.00	Overdue
7	7	42000.00	Paid
8	8	45000.00	Pending

Complaint

id	type	status
1	Electrical	Resolved
2	Plumbing	Pending
3	Furniture	In Progress
4	Electrical	Pending
5	Cleaning	Resolved

Query Execution — Part 1

Simple SELECT and multi-table JOIN

Query 1: Students with pending fees

```
SELECT s.student_id, s.student_name,
       f.amount, f.status
FROM Student s
JOIN Fee_Payment f ON s.student_id = f.student_id
WHERE f.status = 'Pending';
```

id	name	amount	status
2	Priya Nair	45000.00	Pending
4	Sneha Iyer	42000.00	Pending
8	Meera Joshi	45000.00	Pending

Query 2: Occupancy — student, room, block

```
SELECT s.student_name, r.room_no, b.block_name
FROM Student s
JOIN Room r ON s.room_id = r.room_id
JOIN Hostel_Block b ON r.block_id = b.block_id
ORDER BY b.block_name, r.room_no;
```

student_name	room_no	block_name
Aditya Sharma	A-101	Block A
Priya Nair	A-101	Block A
Rahul Gupta	A-102	Block A
Sneha Iyer	B-201	Block B
Karan Mehta	B-201	Block B

Query Execution — Part 2

Aggregation with GROUP BY and HAVING

Query 3: Complaint count by status

```
SELECT status, COUNT(*) AS total
FROM Complaint
GROUP BY status;
```

status	total
In Progress	1
Pending	2
Resolved	2

Query 4: Blocks housing more than 2 students

```
SELECT b.block_name,
       COUNT(s.student_id) AS student_count
FROM Hostel_Block b
JOIN Room r ON b.block_id = r.block_id
JOIN Student s ON s.room_id = r.room_id
GROUP BY b.block_name
HAVING COUNT(s.student_id) > 2;
```

block_name	student_count
Block A	4
Block B	4

Query Execution — Part 3

Data modification: UPDATE and verify

Query 5: Mark a pending payment as paid

```
UPDATE Fee_Payment
SET status = 'Paid', paid_date = '2026-07-20'
WHERE payment_id = 2;

SELECT payment_id, status, paid_date
FROM Fee_Payment WHERE payment_id = 2;
```

payment_id	status	paid_date
2	Paid	2026-07-20

Before: Pending, NULL → After: Paid, 2026-07-20

Next: Presentation 3 — full implementation with a working application layer and live end-to-end demo.

Thank you — questions welcome